

ARCHITECTURE DOCUMENT

Smart Inventory Management System

Project-specific architecture for the Node.js, Express.js, MySQL, and vanilla JavaScript inventory management application.

Project Smart Inventory Management System	Document Type Software Architecture Document
Backend Node.js and Express.js	Database MySQL 8+
Frontend HTML5, CSS3, Vanilla JavaScript	Prepared For Project Submission

1. Project Overview

Smart Inventory Management System is a full-stack inventory and sales management application for retail or warehouse operations. The system supports product stock tracking, FIFO batch-based inventory deduction, supplier management, order processing, customer history, staff access control, analytics dashboards, exports, monitoring, and invoice delivery.

The application is built with a Node.js and Express.js backend, a MySQL database, and a vanilla HTML, CSS, and JavaScript frontend served from the same Express application.

2. Architecture Style

The project follows a layered web application architecture:

- **Presentation Layer:** Browser-based UI in `public/`
- **API Layer:** Express route modules in `routes/`
- **Business Logic Layer:** Inventory, alerts, logging, mail, validation, and movement utilities in `lib/` and `src/shared/`
- **Data Access Layer:** MySQL connection pool and SQL queries using `mysql2`
- **Database Layer:** MySQL schema, migrations, and bootstrapping scripts in `db/`

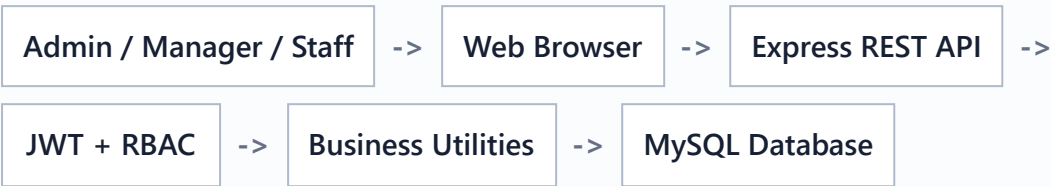
This structure keeps the user interface, HTTP endpoints, reusable business logic, and database concerns separated while remaining practical for a small to medium inventory system.

3. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript, Chart.js
Backend	Node.js, Express.js
Database	MySQL 8+
Authentication	JWT, bcryptjs
Email	nodemailer
Exports	CSV and PDF-style generated reports
Development	nodemon

4. High-Level System Flow

System Request Flow



The Express application also serves static frontend assets and connects to email, export, monitoring, and backup workflows.

5. Main Application Components

5.1 Frontend

The frontend is located in `public/` and is served directly by Express.

File	Responsibility
<code>public/index.html</code>	Main single-page interface
<code>public/app.js</code>	Authentication, navigation, orders, customers, employees, invoices, exports, and shared UI behavior
<code>public/inventory.js</code>	Inventory CRUD, restocking, supplier linking, batch management, and stock workflows
<code>public/suppliers.js</code>	Supplier listing and supplier product intelligence
<code>public/dashboard.js</code>	Sales reports, charts, alerts, approvals, and dashboard metrics
<code>public/styles.css</code> , <code>public/theme.css</code>	UI styling and theme rules

5.2 Backend Application

The backend starts through `server.js` , which loads `src/server.js` . The Express app is created in `src/app.js` .

- Serves static frontend assets
- Parses JSON and form data
- Enables CORS

- Registers API routes
- Provides `/api/health`
- Returns `index.html` for non-API routes
- Handles 404 and centralized errors

5.3 API Route Modules

Module	Base Path	Responsibility
<code>routes/auth.js</code>	<code>/api/auth</code>	Login, current user, logout, password reset/change
<code>routes/inventory.js</code>	<code>/api/inventory</code>	Product CRUD, stock alerts, pending approvals, supplier links, restock, FIFO batches
<code>routes/suppliers.js</code>	<code>/api/suppliers</code>	Supplier CRUD, supplier intelligence, supplier products
<code>routes/orders.js</code>	<code>/api/orders</code>	Order CRUD, FIFO stock deduction/restoration, breakdowns, invoice email
<code>routes/employees.js</code>	<code>/api/employees</code>	User and staff management
<code>routes/customers.js</code>	<code>/api/customers</code>	Customer CRUD and customer order history
<code>routes/exports.js</code>	<code>/api/exports</code>	Daily exports and report downloads
<code>routes/monitoring.js</code>	<code>/api/monitoring</code>	Audit logs, system overview, team health, backups

6. Authentication and Authorization

The system uses JWT-based authentication. After login, the backend validates the submitted password with bcrypt and returns a signed JWT. The frontend sends this token in the `Authorization: Bearer <token>` header. The authentication middleware verifies the token and role guards restrict protected routes.

Role	Access Level
Admin	Full control, user management, approvals, deletes, audits, and backups
Manager	Inventory and supplier operations, restocking, order management, and monitoring
Staff	Daily operations such as viewing inventory and creating orders

7. Database Architecture

The project uses MySQL with a connection pool defined in `db/connection.js` . The main schema is in `db/schema.sql` , while runtime compatibility and additional tables are handled by `db/bootstrap.js` .

Table	Purpose
<code>users</code>	Login accounts, roles, departments, and employee details
<code>suppliers</code>	Supplier profile and performance details
<code>inventory</code>	Product master data and current stock summary
<code>orders</code>	Customer orders and order status
<code>inventory_movements</code>	Stock movement audit trail
<code>supplier_products</code>	Supplier-to-product mapping, pricing, quality, and lead-time data
<code>activity_logs</code>	User and system activity audit records
<code>customers</code>	Customer contact and history support
<code>inventory_batches</code>	FIFO stock batches with purchase cost, selling price, and available quantity
<code>order_batch_usage</code>	Tracks which inventory batches were consumed by each order

8. Entity Relationship View



9. Key Business Workflows

9.1 Inventory Restocking

1. Admin or manager selects a product.
2. User enters supplier, quantity, purchase cost, selling price, and optional expiry date.
3. Backend creates or updates the related inventory batch.
4. Product stock is recalculated from active batches.
5. Inventory status is recalculated using low-stock rules.
6. Stock movement and activity logs are recorded.

9.2 FIFO Order Processing

1. User creates an order for a selected inventory item.
2. Backend checks available stock.
3. Stock is deducted from the oldest available batches first.
4. Batch usage is saved in `order_batch_usage`.
5. Order total cost and profit can be calculated from consumed batch costs.
6. Inventory stock and status are synchronized.
7. Low-stock alerts may be triggered.

9.3 Order Cancellation and Restoration

- 1. Order status changes to cancelled or the order is deleted.
- 2. Backend reads the original batch usage.
- 3. Stock is restored to the same batches that were consumed.
- 4. Inventory summary stock is synchronized.
- 5. Inventory movement and activity logs are updated.

9.4 Dashboard and Monitoring

- 1. Frontend requests analytics from dashboard and monitoring APIs.
- 2. Backend aggregates order, inventory, supplier, audit, and staff data from MySQL.
- 3. Frontend renders summary cards, alerts, charts, pending approvals, and reports.

9.5 Invoice Email

- 1. User requests invoice email for an order.
- 2. Backend validates order details and customer contact information.
- 3. `lib/mailer.js` sends the invoice through nodemailer.
- 4. Activity is logged for tracking.

10. Security Design

Security controls include JWT authentication, role-based authorization, bcrypt password hashing, centralized validation helpers, restricted destructive actions, and activity logs for important operations.

Recommended future hardening includes httpOnly cookie sessions, login rate limiting, wider request validation coverage, transaction wrappers around inventory/order workflows, and secret rotation before deployment.

11. Deployment Architecture



Required environment variables include `PORT`, `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `JWT_EXPIRES_IN`, and SMTP variables if invoice email is enabled.

12. Quality Attributes

Attribute	Project Support
Maintainability	Separate route modules, shared utilities, and frontend modules
Security	JWT, RBAC, bcrypt, protected admin routes
Reliability	Auto-bootstrapping database fixes and stock synchronization
Auditability	Activity logs and inventory movement records
Usability	Single-page dashboard with inventory, orders, suppliers, customers, and reports
Extensibility	Route-per-domain structure allows new modules to be added cleanly

13. Future Enhancements

- Add automated tests for FIFO order creation, cancellation, and restoration.
- Use database transactions for order and restock workflows.
- Add normalized order item support for multi-product orders.
- Add pagination and filtering for large datasets.
- Add formal migration tooling instead of mixed schema plus bootstrapping.
- Add role-specific dashboard views.
- Add REST API documentation with request and response examples.
- Add production deployment guide for cloud hosting.